

ClearSkin Platform - Complete Project Documentation

Version: 1.0

Date: November 6, 2025

Project Type: Mobile-First Web Application (PWA)

Target Launch: Q1 2026

Table of Contents

1. Business Case
 2. Functions and Features
 3. Look, Feel, and UX/UI
 4. Technical Architecture and DevOps
 5. Key Considerations
 6. Working Demo
 7. Next Steps
-

1. Business Case

Problem Statement

The skincare industry is saturated with marketing hype, making it difficult for consumers to understand what products actually contain and whether they will work for their specific skin needs. Current solutions are:

- **Fragmented:** Ingredient databases exist but are disconnected from personal tracking
- **Complex:** Technical ingredient information is difficult for average consumers to understand
- **Not Personalized:** No connection between what worked for you in the past and what might work in the future

Solution

ClearSkin is a mobile-first platform that empowers users to:

- Scan product ingredient lists instantly using their smartphone camera
- Get AI-powered analysis in plain language (not marketing speak)
- Track products they've tried and rate their effectiveness
- Compare products side-by-side based on actual ingredients

- Build a personal skincare profile that learns what works for them

Target Market

- **Primary:** Health-conscious millennials and Gen Z (25-40 years old)
- **Secondary:** Anyone with sensitive skin or specific skin concerns
- **Geographic:** Global (starting with English-speaking markets)

Value Proposition

"Know what's in your skincare. Cut through the marketing hype."

Unlike competitors who focus on selling products or providing databases, ClearSkin is:

1. **Consumer-first:** No affiliate links, no product sales
2. **Personal:** Tracks YOUR experience, not crowdsourced reviews
3. **Transparent:** Plain language explanations of ingredients
4. **Free:** MVP will be free with potential premium features later

Market Opportunity

- Global skincare market: \$177 billion (2023)
- "Clean beauty" segment growing at 9.7% CAGR
- 73% of consumers read ingredient lists but don't understand them
- Average person uses 9-12 skincare products daily

Revenue Model (Future)

Phase 1 (MVP): Free, build user base

Phase 2: Freemium model

- Free: 10 scans/month, basic features
- Premium (\$4.99/month): Unlimited scans, detailed comparisons, recommendations
- Phase 3:** B2B partnerships with dermatologists, skincare brands for transparency badges

2. Functions and Features

2.1 Core Features (MVP)

Landing Page

- Clean, minimal hero section with value proposition

- "Scan Your First Product" CTA
- Three key benefits highlighted:
 - Ingredient Analysis
 - Personal History
 - Smart Recommendations
- Footer disclaimer: "Not medical advice"

Product Scanning & Analysis

User Flow:

1. User taps "Scan Product"
2. Takes photo of ingredient list or uploads from library
3. AI analyzes ingredients (1-2 second processing)
4. Results displayed in structured format

Analysis Output:

- **Quick Summary:** Plain language overview in 2-3 sentences
- **Overall Rating:** AI-generated quality score (1-5 stars)
- **Suited For:** Tags for skin types (e.g., "Sensitive Skin", "Daily Use")
- **Key Ingredients:** Expandable cards showing:
 - Ingredient name
 - Purpose (e.g., "Humectant", "Surfactant")
 - Detailed explanation (expandable)
 - Benefits list
- **Things to Consider:** Potential concerns or warnings
- **Action Buttons:**
 - Save to My Products
 - Rate This Product

Product History ("My Products")

- **Quick Stats Dashboard** (top of page):
 - Total products scanned
 - Average rating

- Number of favorites (4+ stars)
- **Visual Grid Layout:** 2-column grid of product cards
- **Each Card Shows:**
 - Product image (from scan or stock photo)
 - Brand name
 - Product name
 - Star rating
 - Date added
- **Click any card** → View full analysis again

Product Comparison

User Flow:

1. Click "Compare Products" from menu
2. See two side-by-side empty cards
3. For each card, choose to either:
 - **Scan/Upload new product** (camera/upload icon)
 - **Select from saved products** (opens product library)
4. Once both cards filled, comparison insights appear automatically

Comparison Output:

- **Side-by-side product cards** with images, names, ratings
- **Key Insights Section:**
 - **Similarities:** Common ingredients or suitable skin types
 - **Key Differences:** What each product targets
 - **Recommendation:** Which scores higher based on ratings
- **"View Detailed Comparison"** button → Opens category-by-category breakdown:
 - Summary comparison
 - Suited For comparison
 - Key Ingredients comparison
 - Concerns comparison
 - Rating comparison

User Profile

- **Basic Information:**
 - Name
 - Email (for account)
- **Skin Profile** (optional, helps with recommendations):
 - Skin Type: Normal, Dry, Oily, Combination, Sensitive
 - Skin Concerns (multi-select): Acne, Aging, Hyperpigmentation, Redness, Texture, Dryness
- **Settings:**
 - Notification preferences (placeholder)
 - Privacy & Data (placeholder)
 - Terms & Conditions (placeholder)
- **Log Out** option

Navigation

- **Hamburger Menu** (top-right):
 - Home
 - Scan Product
 - My Products
 - Compare Products
 - Profile
- Smooth slide-in animation
- Dark overlay when open
- Auto-closes on navigation

2.2 Future Features (Post-MVP)

Phase 2 Features

- **Smart Recommendations:** "Based on products you loved, try..."
- **Product Search:** Search by name or ingredient
- **Routine Builder:** Morning/evening skincare routines
- **Expiration Tracking:** When to replace products
- **Price Tracking:** Alert when products go on sale

Phase 3 Features

- **Community Ratings:** See aggregate reviews (opt-in)
 - **Dermatologist Verification:** Verified expert opinions
 - **Ingredient Deep Dives:** Educational content about key ingredients
 - **Sustainability Scores:** Environmental impact ratings
 - **Duplicate Detection:** "You already have a similar product"
-

3. Look, Feel, and UX/UI

3.1 Design Philosophy

Inspiration: The Ordinary, The Act, Acid League (minimalist skincare brands)

Core Principles:

1. **Minimal & Clean:** Generous white space, no clutter
2. **Typography-Driven:** Light font weights, clear hierarchy
3. **Functional First:** Utility over decoration
4. **Mobile-Optimized:** Designed for smartphone use
5. **Accessible:** High contrast, readable fonts, clear CTAs

3.2 Visual Design System

Color Palette

Primary:

- Black (#000000) - CTAs, headers, emphasis
- White (#FFFFFF) - Background, cards
- Gray 50 (#FAFAFA) - Section backgrounds
- Gray 100 (#F5F5F5) - Hover states
- Gray 200 (#E5E5E5) - Borders
- Gray 400 (#A3A3A3) - Secondary text
- Gray 600 (#525252) - Body text

Accent:

- Amber 50 (#FFFBEB) - Warning backgrounds
- Amber 200 (#FDE68A) - Warning borders

Typography

- **Font Family:** System font stack (native to each device)
- **Weights:**
 - `font-light` (300) - Body text, descriptions
 - `font-normal` (400) - Headings, labels
- **Sizes:**
 - Hero: 2.5rem (40px) on mobile, 3rem (48px) on desktop
 - H1: 1.5rem (24px)
 - H2: 1.25rem (20px)
 - H3: 1.125rem (18px)
 - Body: 1rem (16px)
 - Small: 0.875rem (14px)
 - Extra Small: 0.75rem (12px)

Spacing System (Tailwind)

- Base unit: 0.25rem (4px)
- Common spacing: 1rem (16px), 1.5rem (24px), 2rem (32px)
- Section padding: 1.5rem (mobile), 2rem (desktop)

Components

Buttons:

- **Primary:** Black background, white text, full-width on mobile
- **Secondary:** White background, black border, hover state
- **Text:** No background, gray text, hover to black

Cards:

- White background
- 1px gray-200 border
- Subtle shadow on hover
- Rounded corners (0.5rem)

Input Fields:

- 1px gray-300 border

- Focus: Black border
- Light font weight
- Padding: 0.75rem vertical, 1rem horizontal

Product Cards:

- Square aspect ratio for images
- Brand name in small, light, uppercase text
- Product name in normal weight
- Star ratings in black (filled) or gray-200 (empty)

3.3 UX Patterns

Progressive Disclosure

- Ingredient details are collapsed by default
- Click to expand for full information
- Prevents information overload

Empty States

- "No products yet" with clear CTA to scan first product
- Helpful illustrations (icon-based)

Loading States

- Show "Analyzing..." when processing images
- 1-2 second typical wait time

Error Handling

- Friendly error messages
- Suggest next actions ("Try taking a clearer photo")

Mobile-First Interactions

- Large tap targets (min 44x44px)
- Swipe gestures considered but not required
- Thumb-friendly bottom navigation via hamburger menu

Information Hierarchy

- Summary first (TL;DR)
 - Details available but optional
 - Most important info above the fold
-

4. Technical Architecture and DevOps

4.1 Technology Stack

Frontend

- **Framework:** Next.js 14+ (React)
 - App Router (not Pages Router)
 - Server Components where possible
 - Client Components for interactivity
- **Language:** TypeScript
- **Styling:** Tailwind CSS v3
- **State Management:**
 - React Context for user session
 - Zustand (lightweight) for global state if needed
- **Image Handling:** next/image for optimization
- **PWA:** next-pwa plugin for installability

Backend

- **API:** Next.js API Routes (serverless functions)
- **Authentication:** Supabase Auth
 - Google OAuth
 - Email/Password
 - Magic Links
- **Database:** Supabase (PostgreSQL)
- **File Storage:** AWS S3
- **AI Provider:** Google Gemini 1.5 Flash API

DevOps & Deployment

- **Hosting:** Vercel

- **CI/CD:** GitHub Actions → Vercel
- **Monitoring:** Vercel Analytics + Sentry (errors)
- **Version Control:** Git + GitHub

4.2 Database Schema (Supabase PostgreSQL)

sql

-- Users table (managed by Supabase Auth, extended)

```
CREATE TABLE user_profiles (  
  id UUID PRIMARY KEY REFERENCES auth.users(id),  
  name TEXT,  
  skin_type TEXT CHECK (skin_type IN ('Normal', 'Dry', 'Oily', 'Combination', 'Sensitive')),  
  skin_concerns TEXT[], -- Array of concerns  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

-- Products table

```
CREATE TABLE products (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  name TEXT NOT NULL,  
  brand TEXT,  
  image_url TEXT, -- S3 URL  
  date_added TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

-- Ingredient analysis table

```
CREATE TABLE ingredient_analyses (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,  
  summary TEXT NOT NULL,  
  overall_rating DECIMAL(2,1) CHECK (overall_rating >= 0 AND overall_rating <= 5),  
  suited_for TEXT[], -- Array of skin types/uses  
  key_ingredients JSONB NOT NULL, -- Array of {name, purpose, detail, benefits}  
  concerns TEXT[], -- Array of concern strings  
  raw_ai_response JSONB, -- Store full AI response for debugging  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

-- User ratings table

```
CREATE TABLE user_ratings (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,  
  rating INTEGER CHECK (rating >= 1 AND rating <= 5),  
  notes TEXT,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  UNIQUE(user_id, product_id) -- One rating per user per product  
);
```

-- Indexes for performance

```
CREATE INDEX idx_products_user_id ON products(user_id);
CREATE INDEX idx_products_date_added ON products(date_added DESC);
CREATE INDEX idx_ingredient_analyses_product_id ON ingredient_analyses(product_id);
CREATE INDEX idx_user_ratings_user_id ON user_ratings(user_id);
CREATE INDEX idx_user_ratings_product_id ON user_ratings(product_id);
```

-- Row Level Security (RLS) policies

```
ALTER TABLE user_profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE products ENABLE ROW LEVEL SECURITY;
ALTER TABLE ingredient_analyses ENABLE ROW LEVEL SECURITY;
ALTER TABLE user_ratings ENABLE ROW LEVEL SECURITY;
```

-- Users can only see their own data

```
CREATE POLICY "Users can view own profile" ON user_profiles FOR SELECT USING (auth.uid() = id);
CREATE POLICY "Users can update own profile" ON user_profiles FOR UPDATE USING (auth.uid() = id);

CREATE POLICY "Users can view own products" ON products FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "Users can insert own products" ON products FOR INSERT WITH CHECK (auth.uid() = user_id);
CREATE POLICY "Users can delete own products" ON products FOR DELETE USING (auth.uid() = user_id);

CREATE POLICY "Users can view analyses for own products" ON ingredient_analyses FOR SELECT
  USING (product_id IN (SELECT id FROM products WHERE user_id = auth.uid()));

CREATE POLICY "Users can view own ratings" ON user_ratings FOR SELECT USING (auth.uid() = user_id);
CREATE POLICY "Users can manage own ratings" ON user_ratings FOR ALL USING (auth.uid() = user_id);
```

4.3 API Architecture

API Endpoints (Next.js API Routes)

POST /api/analyze

- └─ Input: FormData with image file
- └─ Process:
 - | 1. Validate file (type, size < 5MB)
 - | 2. Upload to S3
 - | 3. Call Gemini API with image URL + prompt
 - | 4. Parse AI response into structured format
 - | 5. Save to products + ingredient_analyses tables
 - | 6. Return analysis JSON
- └─ Output: { productId, analysis }

GET /api/products

- └─ Query params: ?sortBy=date&rating&limit=20
- └─ Process: Fetch user's products with analyses
- └─ Output: { products: [...] }

GET /api/products/[id]

- └─ Process: Fetch single product with full analysis
- └─ Output: { product, analysis }

POST /api/products/[id]/rate

- └─ Input: { rating: 1-5, notes?: string }
- └─ Process: Upsert user_ratings table
- └─ Output: { success: true }

DELETE /api/products/[id]

- └─ Process: Delete product (cascades to analyses and ratings)
- └─ Output: { success: true }

POST /api/compare

- └─ Input: { productIds: [id1, id2] }
- └─ Process: Fetch both products with analyses, generate comparison
- └─ Output: { comparison: {...} }

GET /api/user/profile

- └─ Process: Fetch user profile data
- └─ Output: { profile: {...} }

PUT /api/user/profile

- └─ Input: { name?, skinType?, skinConcerns? }
- └─ Process: Update user_profiles table
- └─ Output: { success: true }

Gemini AI Integration

Prompt Template:

Analyze the skincare product ingredients in this image. Return a JSON response with:

```
{
  "product_name": "Extract product name if visible",
  "summary": "2-3 sentence plain language summary of what this product does",
  "overall_rating": 4.2, // Quality score 1-5 based on ingredient quality
  "suited_for": ["All Skin Types", "Sensitive Skin"], // Array of skin types
  "key_ingredients": [
    {
      "name": "Ingredient name",
      "purpose": "One-word purpose (e.g., Humectant, Surfactant)",
      "detail": "1-2 sentence explanation in plain language",
      "benefits": ["Benefit 1", "Benefit 2"]
    }
  ],
  "concerns": ["Concern 1", "Concern 2"] // Potential issues like fragrance, allergens
}
```

Focus on being accurate, concise, and consumer-friendly. Avoid marketing language.

API Call (Node.js):

```
typescript

import { GoogleGenerativeAI } from "@google/generative-ai";

const genAI = new GoogleGenerativeAI(process.env.GEMINI_API_KEY);

async function analyzeIngredients(imageUrl: string) {
  const model = genAI.getGenerativeModel({ model: "gemini-1.5-flash" });

  const result = await model.generateContent([
    {
      inlineData: {
        mimeType: "image/jpeg",
        data: await fetchImageAsBase64(imageUrl)
      }
    },
    { text: PROMPT_TEMPLATE }
  ]);

  const response = result.response.text();
  return JSON.parse(response);
}
```

4.4 File Structure

```
clearskin/
├── .env.local           # Environment variables (not committed)
├── .gitignore
├── next.config.js       # Next.js configuration
├── tailwind.config.js   # Tailwind configuration
├── tsconfig.json        # TypeScript configuration
├── package.json
├── public/
│   ├── icons/           # PWA icons
│   └── manifest.json     # PWA manifest
├── src/
│   ├── app/             # Next.js App Router
│   │   ├── layout.tsx    # Root layout with providers
│   │   ├── page.tsx      # Landing page
│   │   ├── scan/
│   │   │   └── page.tsx   # Scan product page
│   │   ├── results/
│   │   │   └── [id]/
│   │   │       └── page.tsx # Analysis results page
│   │   ├── products/
│   │   │   └── page.tsx   # Product history page
│   │   ├── compare/
│   │   │   └── page.tsx   # Compare products page
│   │   ├── profile/
│   │   │   └── page.tsx   # User profile page
│   │   └── api/          # API routes
│   │       ├── analyze/
│   │       │   └── route.ts # POST analyze image
│   │       ├── products/
│   │       │   ├── route.ts # GET user products
│   │       │   └── [id]/
│   │       │       ├── route.ts # GET/DELETE product
│   │       │       └── rate/
│   │       │           └── route.ts # POST rating
│   │       ├── compare/
│   │       │   └── route.ts # POST compare
│   │       └── user/
│   │           ├── profile/
│   │           │   └── route.ts # GET/PUT profile
│   └── components/       # React components
│       ├── layout/
│       │   ├── HamburgerMenu.tsx
│       │   ├── Header.tsx
│       │   └── Footer.tsx
```

```

| | | └─ products/
| | |   | └─ ProductCard.tsx
| | |   | └─ ProductGrid.tsx
| | |   | └─ ProductStats.tsx
| | | └─ analysis/
| | |   | └─ IngredientCard.tsx
| | |   | └─ SummarySection.tsx
| | |   | └─ ConcernsSection.tsx
| | | └─ comparison/
| | |   | └─ ComparisonCard.tsx
| | |   | └─ ComparisonSlot.tsx
| | |   | └─ InsightsSection.tsx
| | | └─ ui/           # Reusable UI components
| | |   | └─ Button.tsx
| | |   | └─ Card.tsx
| | |   | └─ Input.tsx
| | |   | └─ StarRating.tsx
| | | └─ providers/
| | |   | └─ AuthProvider.tsx # Supabase auth context
| | └─ lib/           # Utility functions
| |   | └─ supabase/
| |   |   | └─ client.ts    # Browser client
| |   |   | └─ server.ts    # Server client
| |   |   | └─ types.ts     # Database types
| |   | └─ s3/
| |   |   | └─ upload.ts    # S3 upload utilities
| |   | └─ ai/
| |   |   | └─ gemini.ts    # Gemini API wrapper
| |   |   | └─ utils.ts     # General utilities
| | └─ types/
| |   | └─ index.ts        # TypeScript type definitions
| |   | └─ styles/
| |   |   | └─ globals.css  # Global styles + Tailwind imports
| | └─ supabase/
| |   | └─ migrations/     # Database migrations
| |   |   | └─ 001_initial_schema.sql
| |   | └─ config.toml     # Supabase configuration

```

4.5 Environment Variables

```
bash
```


Supabase

NEXT_PUBLIC_SUPABASE_URL=https://your-project.supabase.co

NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key

SUPABASE_SERVICE_ROLE_KEY=your-service-role-key

AWS S3

AWS_REGION=us-east-1

AWS_ACCESS_KEY_ID=your-access-key

AWS_SECRET_ACCESS_KEY=your-secret-key

AWS_S3_BUCKET=clearskin-ingredient-photos

Google Gemini

GEMINI_API_KEY=your-gemini-api-key

App

NEXT_PUBLIC_APP_URL=https://clearskin.app

4.6 Deployment Pipeline

Developer Workflow:

- └─ 1. Local Development
 - | └─ npm run dev (localhost:3000)
 - | └─ Supabase local setup (optional)
 - | └─ Connect to dev Supabase instance
- └─ 2. Commit & Push
 - | └─ git add .
 - | └─ git commit -m "feature: ..."
 - | └─ git push origin feature-branch
- └─ 3. Pull Request
 - | └─ GitHub PR created
 - | └─ Vercel Preview Deploy triggered
 - | └─ Review changes on preview URL
- └─ 4. Merge to Main
 - | └─ PR merged to main branch
 - | └─ Vercel Production Deploy triggered
- └─ 5. Live on clearskin.app

Vercel Configuration:

- └─ Build Command: npm run build
- └─ Output Directory: .next
- └─ Install Command: npm install
- └─ Node Version: 20.x
- └─ Environment Variables: Set in Vercel dashboard

4.7 Cost Breakdown (Monthly)

Development/MVP Stage:

- Vercel: \$0 (Free tier: 100GB bandwidth, unlimited deploys)
- Supabase: \$0 (Free tier: 500MB database, 1GB storage, 2GB file storage)
- AWS S3: \$1-5 (depends on photos, ~\$0.023/GB storage + \$0.0004/1000 GET requests)
- Gemini API: \$0.50-2 (1000-10000 scans at ~\$0.0001875/scan)
- Domain (clearskin.app): \$12/year (~\$1/month)

Total MVP: \$2.50-8/month 🎉

At Scale (10,000 users, 50,000 scans/month):

- Vercel: \$20/month (Pro plan for better performance)
- Supabase: \$25/month (Pro plan: 8GB database, 100GB storage)
- AWS S3: \$20-30/month
- Gemini API: \$10-15/month
- CDN (Cloudflare): \$5-10/month

Total at scale: \$80-100/month

5. Key Considerations

5.1 Legal & Compliance

Disclaimers

- **Critical:** Display "Not medical advice. Consult a dermatologist for skin concerns." on:
 - Landing page footer
 - Analysis results page
 - Comparison page
 - Terms of Service
- **Liability:** Make it clear the platform is educational, not prescriptive

Terms of Service & Privacy Policy

Must include:

- How user data is collected and stored
- Image retention policy (delete after 30 days recommended)

- User rights (GDPR, CCPA compliance)
- AI-generated content disclaimer
- User-generated ratings policy

GDPR Compliance (EU users)

- Cookie consent banner
- Right to be forgotten (delete account + all data)
- Data export functionality
- Clear privacy policy

Medical Device Regulations

- ClearSkin is NOT a medical device (no diagnosis or treatment)
- Avoid language like "treats", "cures", "diagnoses"
- Use "suitable for", "may help with", "commonly used for"

5.2 AI Considerations

Accuracy & Reliability

- **AI Hallucination Risk:** Gemini may occasionally misread ingredients or provide incorrect information
- **Mitigation:**
 - Store raw AI responses for debugging
 - Add "Report incorrect analysis" button
 - Manual review for flagged products
 - Confidence scores in future iterations

Prompt Engineering

- Iterate on prompts based on real results
- A/B test different prompt variations
- Fine-tune for consistency

Rate Limiting

- Gemini API has rate limits
- Implement user-level throttling (10 scans/hour for free users)
- Queue system for high traffic

Fallback Handling

- If AI fails, show user-friendly error: "We couldn't analyze this image. Please try again with better lighting."
- Log failures for monitoring

5.3 Performance & Scalability

Image Upload Optimization

- Compress images client-side before upload (reduce size 50-70%)
- Lazy load product images in history grid
- Use next/image for automatic optimization

Database Performance

- Indexes on frequently queried columns (user_id, date_added)
- Paginate product history (20 per page)
- Cache frequent queries (user profile)

API Response Times

- Target: < 2 seconds for ingredient analysis
- Use Vercel Edge Functions for faster cold starts
- Implement loading states to manage user expectations

Caching Strategy

- Cache analysis results indefinitely (ingredients don't change)
- Cache user product list for 5 minutes
- Use SWR (stale-while-revalidate) for smooth UX

5.4 Security

Authentication

- Use Supabase Auth (battle-tested, secure)
- Implement email verification for new accounts
- Rate limit login attempts

API Security

- Validate all inputs server-side

- Use CORS properly (only allow your domain)
- Never expose API keys in client code
- Implement CSRF protection

Image Upload Security

- Validate file types (only JPG, PNG, WebP)
- Max file size: 5MB
- Scan for malicious content (if possible)
- Generate unique filenames (prevent overwrites)

Row Level Security (RLS)

- Supabase RLS ensures users only see their own data
- Test RLS policies thoroughly

5.5 User Experience

Onboarding

- First-time users should see a brief tutorial
- "Scan your first product" should be prominent
- Skip authentication for first scan (let users try it)
- Prompt to create account after first analysis

Offline Functionality

- PWA allows offline browsing of saved products
- Queue scans when offline, process when online (future)

Accessibility

- WCAG 2.1 AA compliance
- Keyboard navigation support
- Screen reader friendly (semantic HTML)
- High contrast mode support
- Alt text for all images

Mobile Considerations

- Optimize for one-handed use

- Large tap targets (min 44x44px)
- Bottom-aligned CTAs for thumb reach
- Smooth animations (60fps)

5.6 Content Moderation

Inappropriate Images

- Users might upload non-skincare images
- **Mitigation:**
 - AI pre-check: "Is this an ingredient list?"
 - Manual review queue for flagged images
 - User reporting system

Spam Prevention

- Rate limiting on scans
- CAPTCHA for signup (if needed)
- Monitor for abuse patterns

5.7 Testing Strategy

Unit Tests

- Test utility functions (lib/)
- Test API route logic
- Test component rendering

Integration Tests

- Test full API workflows (scan → analyze → save)
- Test authentication flows
- Test database queries

E2E Tests (Playwright or Cypress)

- Test critical user journeys:
 - Signup → Scan → View results → Save product
 - Login → View history → Compare products
 - Update profile → Save preferences

Manual Testing

- Test on real devices (iOS Safari, Android Chrome)
 - Test with various lighting conditions for photos
 - Test edge cases (blurry images, no ingredients visible)
-

6. Working Demo

A fully functional prototype has been created demonstrating all core features:

Demo URL: [Embedded in this conversation as an interactive artifact]

What the Demo Includes:

Implemented Pages:

1. Landing Page

- Clean hero section
- Three value propositions
- "Scan Your First Product" CTA
- Footer disclaimer

2. Scan/Upload Page

- Camera/upload interface
- Tips for best results
- Simulated scanning (click "Choose Photo" to see results)

3. Analysis Results Page

- Product name and overall rating
- Quick summary section
- Suited For tags
- Expandable ingredient cards with:
 - Ingredient name and purpose
 - Detailed explanation
 - Benefits list
- Things to Consider section
- Save and Rate buttons

4. Product History Page

- Quick stats (total products, avg rating, favorites)
- 2-column visual grid
- Product cards with images, ratings, dates
- Click any card to view full analysis

5. Compare Products Page

- Two side-by-side comparison slots
- Each slot allows:
 - Scan/Upload new product photo
 - Select from saved products
- Once both filled, shows:
 - Side-by-side product cards
 - Key Insights (similarities, differences, recommendation)
 - "View Detailed Comparison" button
- Remove product functionality

6. Profile Page

- Basic information fields (name, email)
- Skin Type selector (5 options)
- Skin Concerns multi-select (6 options)
- Settings placeholders
- Save Changes button
- Log Out option

Navigation & UX:

- Hamburger menu (top-right) with smooth animations
- All pages interconnected
- Mobile-first responsive design
- Consistent minimalist aesthetic
- Expandable/collapsible sections
- Loading state simulations

Design Implementation:

- Tailwind CSS utility classes
- Light font weights throughout
- Generous white space
- Black and white color scheme
- Gray scale for secondary elements
- Star rating system (filled/unfilled)
- Card-based layouts
- Hover states and transitions

Demo Limitations (Simulated):

- **No real authentication:** Uses local state only
- **No real AI analysis:** Shows mock data
- **No real image upload:** Clicking upload shows simulated results
- **No persistence:** Refresh resets everything
- **Mock data:** 3 pre-populated products for demonstration

How to Test the Demo:

1. **Landing Page:** Click "Scan Your First Product"
2. **Scan Page:** Click "Choose Photo" to simulate scanning
3. **Results:** Explore expandable ingredient cards, view summary
4. **History:** Open hamburger menu → "My Products" to see grid
5. **Compare:** Menu → "Compare Products" → Click "Scan/Upload Photo" on both cards
6. **Profile:** Menu → "Profile" → Select skin type and concerns

Demo Code:

The complete React component code is available in the artifact titled "Skincare Ingredient Analyzer" in this conversation. It includes:

- All 7 pages fully functional
- Complete component structure
- All interactive states
- Mock data structures
- Responsive layouts

- All UI patterns implemented

To use this demo code as a starting point:

1. Copy the artifact code
 2. Set up a Next.js project
 3. Install dependencies (lucide-react for icons)
 4. Replace mock data with real API calls
 5. Add Supabase authentication
 6. Integrate Gemini AI API
 7. Connect AWS S3 for image storage
-

7. Next Steps

Phase 1: Foundation (Weeks 1-2)

Week 1: Project Setup

Day 1-2: Infrastructure Setup

- ☐ Create GitHub repository
- ☐ Initialize Next.js 14 project with TypeScript
- ☐ Install dependencies (Tailwind, Supabase, AWS SDK, Gemini)
- ☐ Set up Supabase project
- ☐ Create AWS S3 bucket
- ☐ Obtain Gemini API key
- ☐ Configure environment variables
- ☐ Set up Vercel project and connect to GitHub

Day 3-4: Database & Authentication

- ☐ Run database migration script (create tables)
- ☐ Set up Row Level Security policies
- ☐ Configure Supabase Auth (Google OAuth + Email/Password)
- ☐ Create AuthProvider component
- ☐ Test authentication flow locally

Day 5-7: Core Utilities

- ☐ Implement Supabase client utilities (lib/supabase/)
- ☐ Implement S3 upload utilities (lib/s3/)

- ☐ Implement Gemini AI wrapper (lib/ai/)
- ☐ Create TypeScript types for database models
- ☐ Write unit tests for utilities

Week 2: Core Features - Part 1

Day 1-3: Landing & Scan Pages

- ☐ Build Landing page component (port from demo)
- ☐ Build Scan page with real file upload
- ☐ Implement image compression client-side
- ☐ Test camera access on mobile devices
- ☐ Add loading states and error handling

Day 4-7: Analysis & Results

- ☐ Build API route: POST /api/analyze
- ☐ Integrate S3 upload in API
- ☐ Integrate Gemini AI analysis
- ☐ Parse and validate AI responses
- ☐ Save to database (products + ingredient_analyses)
- ☐ Build Results page component
- ☐ Test end-to-end scan → analyze → display flow

Phase 2: User Features (Weeks 3-4)

Week 3: Product Management

Day 1-3: Product History

- ☐ Build API route: GET /api/products
- ☐ Build Product History page
- ☐ Implement product grid with real data
- ☐ Add statistics calculation
- ☐ Implement pagination (20 per page)
- ☐ Test loading and empty states

Day 4-5: Product Rating

- ☐ Build API route: POST /api/products/[id]/rate
- ☐ Add rating UI to Results page
- ☐ Implement star rating component
- ☐ Save ratings to database
- ☐ Update product display with user ratings

Day 6-7: Profile Management

- ☐ Build API routes: GET/PUT /api/user/profile
- ☐ Build Profile page component
- ☐ Implement skin type selector
- ☐ Implement concerns multi-select
- ☐ Save profile data to database

Week 4: Comparison & Polish

Day 1-3: Product Comparison

- ☐ Build API route: POST /api/compare
- ☐ Build Compare page with dual upload slots
- ☐ Implement "Select from Saved Products" modal
- ☐ Generate comparison insights
- ☐ Build detailed comparison view
- ☐ Test comparison logic

Day 4-7: Polish & Testing

- ☐ Add hamburger menu navigation
- ☐ Implement all loading states
- ☐ Add error boundaries
- ☐ Write integration tests
- ☐ Test on real mobile devices (iOS + Android)
- ☐ Fix responsive design issues
- ☐ Optimize performance (Lighthouse score > 90)

Phase 3: Launch Preparation (Week 5-6)

Week 5: Testing & Refinement

Day 1-2: Security Audit

- ☐ Review all API routes for security issues
- ☐ Test RLS policies in Supabase
- ☐ Implement rate limiting on AI calls
- ☐ Test authentication edge cases
- ☐ Add CSRF protection

Day 3-4: Content & Legal

- ☐ Write Terms of Service
- ☐ Write Privacy Policy
- ☐ Add disclaimers to all relevant pages
- ☐ Create FAQ page

- ☐ Add "Report Incorrect Analysis" feature

Day 5-7: User Testing

- ☐ Recruit 10-15 beta testers
- ☐ Conduct user testing sessions
- ☐ Gather feedback on UX/UI
- ☐ Identify pain points
- ☐ Make improvements based on feedback

Week 6: Launch

Day 1-3: Pre-Launch

- ☐ Set up production environment variables in Vercel
- ☐ Configure custom domain (clearskin.app)
- ☐ Set up SSL certificate
- ☐ Configure email service (for auth emails)
- ☐ Set up error monitoring (Sentry)
- ☐ Set up analytics (Vercel Analytics + Plausible)
- ☐ Final testing on production

Day 4: Launch Day

- ☐ Merge to production
- ☐ Deploy to Vercel
- ☐ Verify all features work in production
- ☐ Monitor error logs closely
- ☐ Test on multiple devices
- ☐ Share with beta testers first

Day 5-7: Post-Launch Monitoring

- ☐ Monitor server errors and fix immediately
- ☐ Track user signups and engagement
- ☐ Monitor AI API costs
- ☐ Gather user feedback
- ☐ Plan iteration based on feedback

Phase 4: Growth & Iteration (Ongoing)

Month 2: User Acquisition

- ☐ Launch on Product Hunt
- ☐ Share on Reddit (r/SkincareAddiction)
- ☐ Create Instagram/TikTok presence

- ☐ Write blog posts about ingredient analysis
- ☐ SEO optimization (meta tags, sitemap)
- ☐ Reach 100 active users

Month 3: Feature Expansion

- ☐ Implement smart recommendations based on history
- ☐ Add product search functionality
- ☐ Implement routine builder
- ☐ Add community ratings (optional)
- ☐ Improve AI prompt based on user feedback
- ☐ Reach 500 active users

Month 4-6: Monetization Preparation

- ☐ Analyze usage patterns
- ☐ Design premium tier features
- ☐ Implement payment system (Stripe)
- ☐ Create subscription tiers
- ☐ A/B test pricing
- ☐ Launch premium features
- ☐ Reach 1,000 active users, 50 paying subscribers

Immediate Action Items (This Week)

For You (Non-Technical Founder):

- 1. Register domain:** Buy clearskin.app (or alternative)
- 2. Create accounts:**
 - GitHub account (for code repository)
 - Vercel account (for hosting)
 - Supabase account (for database)
 - AWS account (for image storage)
 - Google Cloud account (for Gemini API)
- 3. Legal prep:** Research ToS and Privacy Policy templates
- 4. Market research:** Join r/SkincareAddiction, follow skincare influencers
- 5. Content planning:** Start drafting blog posts about ingredients

For Developer (Technical Implementation):

- 1. Review this document thoroughly**

2. **Clone the demo code** from this conversation

3. **Set up development environment:**

```
bash

npx create-next-app@latest clearskin --typescript --tailwind --app
cd clearskin
npm install @supabase/supabase-js @supabase/auth-helpers-nextjs
npm install @google/generative-ai
npm install @aws-sdk/client-s3 @aws-sdk/s3-request-presigner
npm install lucide-react zustand
```

4. **Initialize Supabase:**

- Create new project
- Run database migration script (from Section 4.2)
- Copy connection strings

5. **Set up AWS S3:**

- Create bucket: clearskin-ingredient-photos
- Set up CORS policy
- Create IAM user with S3 access

6. **Get Gemini API key:**

- Go to Google AI Studio
- Create new API key

7. **Configure .env.local** with all credentials

8. **Start with authentication:** Get login/signup working first

9. **Build one feature at a time:** Follow Phase 1 checklist

Resources & Documentation

Technical Documentation:

- Next.js: <https://nextjs.org/docs>
- Supabase: <https://supabase.com/docs>
- Gemini API: <https://ai.google.dev/docs>
- AWS S3: <https://docs.aws.amazon.com/s3/>
- Tailwind CSS: <https://tailwindcss.com/docs>

Community & Support:

- Next.js Discord: <https://discord.gg/nextjs>
- Supabase Discord: <https://discord.supabase.com>
- React subreddit: r/reactjs

Similar Projects for Inspiration:

- INCI Decoder (ingredient lookup)
- Think Dirty (product scanner)
- Yuka (food/cosmetic scanner)
- CosDNA (cosmetic analysis database)

Business Resources:

- Y Combinator Startup School (free course)
- Indie Hackers community
- r/SaaS subreddit

Success Metrics

MVP Success Criteria (Month 1):

-  100+ signups
-  500+ products scanned
-  80% of scans produce accurate results
-  Average session time > 3 minutes
-  < 5% error rate on analysis
-  Mobile responsive on iOS and Android
-  Page load time < 2 seconds

Growth Metrics (Month 3):

-  1,000+ active users
-  5,000+ products scanned
-  30% return rate (users coming back)
-  Average 3+ products per user
-  4.5+ star rating in user feedback

- 🎯 10% conversion to email list

Monetization Metrics (Month 6):

- 💰 2,000+ active users
- 💰 100+ paying subscribers
- 💰 \$500/month recurring revenue
- 💰 < \$200/month operating costs
- 💰 Positive unit economics

Risk Mitigation

Technical Risks:

Risk	Impact	Mitigation
AI inaccuracy	High	Manual review queue, user reporting, confidence scores
API rate limits	Medium	Implement queuing, caching, rate limiting
Storage costs balloon	Medium	Delete old images, compress images, set usage alerts
Database performance	Low	Proper indexing, pagination, caching

Business Risks:

Risk	Impact	Mitigation
Low user adoption	High	Beta testing, iterate on feedback, strong value prop
Legal issues	High	Clear disclaimers, proper ToS/Privacy, avoid medical claims
Competition	Medium	Focus on UX and personalization, move fast
Negative reviews	Medium	Excellent customer support, iterate quickly on bugs

Final Checklist Before Launch

Technical:

- ☐ All features tested on production
- ☐ Error monitoring set up (Sentry)
- ☐ Analytics installed (Vercel + Plausible)
- ☐ Database backups configured
- ☐ Rate limiting implemented
- ☐ Security audit passed
- ☐ Performance optimized (Lighthouse > 90)
- ☐ Mobile tested on real devices

Legal:

- ☐ Terms of Service published
- ☐ Privacy Policy published
- ☐ Cookie consent banner (if needed)
- ☐ Disclaimers on all analysis pages
- ☐ GDPR compliance checked

Content:

- ☐ FAQ page created
- ☐ About page explaining the mission
- ☐ Contact/Support email set up
- ☐ Social media accounts created
- ☐ Launch announcement drafted

Marketing:

- ☐ Product Hunt page prepared
 - ☐ Reddit posts drafted
 - ☐ Email to beta testers ready
 - ☐ Screenshots and demo video created
 - ☐ Press kit prepared (if targeting media)
-

Appendix A: Glossary

Technical Terms:

- **API (Application Programming Interface):** Way for software to communicate with other software
- **PWA (Progressive Web App):** Web app that works like a native app, can be installed on phone
- **RLS (Row Level Security):** Database security ensuring users only see their own data
- **S3 (Simple Storage Service):** Amazon's cloud storage for files/images
- **CDN (Content Delivery Network):** Distributed servers that deliver content faster
- **JWT (JSON Web Token):** Secure way to transmit information between parties

Skincare Terms:

- **INCI:** International Nomenclature of Cosmetic Ingredients (standardized ingredient names)
- **Humectant:** Ingredient that attracts and retains moisture (e.g., Glycerin)
- **Surfactant:** Cleansing agent that removes dirt and oil
- **Emollient:** Ingredient that softens and moisturizes skin
- **Active Ingredient:** Ingredient that produces the main effect (e.g., Retinol for anti-aging)

Appendix B: Sample AI Prompts

Main Analysis Prompt

You are analyzing a skincare product's ingredient list. Be accurate, objective, and consumer-friendly.

Analyze the ingredients in this image and return ONLY a JSON response with this exact structure:

```
{
  "product_name": "Extract product name if visible, otherwise 'Unknown Product'",
  "summary": "2-3 sentences in simple language explaining what this product does and who it's for",
  "overall_rating": 4.2,
  "suited_for": ["Skin Type 1", "Skin Type 2", "Use Case"],
  "key_ingredients": [
    {
      "name": "Ingredient Name",
      "purpose": "One word: Humectant, Surfactant, Emollient, etc.",
      "detail": "1-2 sentences explaining what this does in plain English",
      "benefits": ["Benefit 1", "Benefit 2", "Benefit 3"]
    }
  ],
  "concerns": ["Potential concern 1", "Potential concern 2"]
}
```

Guidelines:

- overall_rating: 1-5 scale based on ingredient quality (4+ means high quality)
- suited_for: Include skin types (Normal, Dry, Oily, Combination, Sensitive) and use cases (Daily Use, Anti-Aging, Acne Treatment, etc.)
- key_ingredients: Include 3-5 most important active ingredients
- concerns: List potential issues like allergens, irritants, fragrance, essential oils, high pH, etc.
- Use simple language, avoid technical jargon
- Be objective, don't use marketing language
- If you can't read ingredients clearly, say so in summary

Comparison Prompt

Compare these two skincare products based on their ingredients:

Product 1: [Product 1 data]

Product 2: [Product 2 data]

Return JSON:

```
{
  "similarities": "What do both products have in common? (ingredients, purposes, skin types)",
```

```
"key_differences": [  
  "Product 1: [key differentiator]",  
  "Product 2: [key differentiator]"  
],  
"recommendation": "Which product is better quality and why? Be objective."  
}
```

Appendix C: Database Migration Script

File: `supabase/migrations/001_initial_schema.sql`

sql

-- Enable UUID extension

```
CREATE EXTENSION IF NOT EXISTS "uuid-oss";
```

-- User Profiles (extends Supabase auth.users)

```
CREATE TABLE user_profiles (  
  id UUID PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,  
  name TEXT,  
  skin_type TEXT CHECK (skin_type IN ('Normal', 'Dry', 'Oily', 'Combination', 'Sensitive')),  
  skin_concerns TEXT[] DEFAULT '{}',  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

-- Products

```
CREATE TABLE products (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  name TEXT NOT NULL,  
  brand TEXT,  
  image_url TEXT,  
  date_added TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

-- Ingredient Analyses

```
CREATE TABLE ingredient_analyses (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,  
  summary TEXT NOT NULL,  
  overall_rating DECIMAL(2,1) CHECK (overall_rating >= 0 AND overall_rating <= 5),  
  suited_for TEXT[] DEFAULT '{}',  
  key_ingredients JSONB NOT NULL DEFAULT '[]',  
  concerns TEXT[] DEFAULT '{}',  
  raw_ai_response JSONB,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()  
);
```

-- User Ratings

```
CREATE TABLE user_ratings (  
  id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),  
  user_id UUID NOT NULL REFERENCES auth.users(id) ON DELETE CASCADE,  
  product_id UUID NOT NULL REFERENCES products(id) ON DELETE CASCADE,  
  rating INTEGER CHECK (rating >= 1 AND rating <= 5),  
  notes TEXT,  
  created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),  
  updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
```

```
    UNIQUE(user_id, product_id)
);

-- Indexes
CREATE INDEX idx_products_user_id ON products(user_id);
CREATE INDEX idx_products_date_added ON products(date_added DESC);
CREATE INDEX idx_ingredient_analyses_product_id ON ingredient_analyses(product_id);
CREATE INDEX idx_user_ratings_user_id ON user_ratings(user_id);
CREATE INDEX idx_user_ratings_product_id ON user_ratings(product_id);

-- Enable Row Level Security
ALTER TABLE user_profiles ENABLE ROW LEVEL SECURITY;
ALTER TABLE products ENABLE ROW LEVEL SECURITY;
ALTER TABLE ingredient_analyses ENABLE ROW LEVEL SECURITY;
ALTER TABLE user_ratings ENABLE ROW LEVEL SECURITY;

-- RLS Policies for user_profiles
CREATE POLICY "Users can view own profile"
ON user_profiles FOR SELECT
USING (auth.uid() = id);

CREATE POLICY "Users can insert own profile"
ON user_profiles FOR INSERT
WITH CHECK (auth.uid() = id);

CREATE POLICY "Users can update own profile"
ON user_profiles FOR UPDATE
USING (auth.uid() = id);

-- RLS Policies for products
CREATE POLICY "Users can view own products"
ON products FOR SELECT
USING (auth.uid() = user_id);

CREATE POLICY "Users can insert own products"
ON products FOR INSERT
WITH CHECK (auth.uid() = user_id);

CREATE POLICY "Users can update own products"
ON products FOR UPDATE
USING (auth.uid() = user_id);

CREATE POLICY "Users can delete own products"
ON products FOR DELETE
USING (auth.uid() = user_id);

-- RLS Policies for ingredient_analyses
```

```
CREATE POLICY "Users can view analyses for own products"
ON ingredient_analyses FOR SELECT
USING (
    product_id IN (
        SELECT id FROM products WHERE user_id = auth.uid()
    )
);
```

```
CREATE POLICY "Service role can insert analyses"
ON ingredient_analyses FOR INSERT
WITH CHECK (true);
```

-- RLS Policies for user_ratings

```
CREATE POLICY "Users can view own ratings"
ON user_ratings FOR SELECT
USING (auth.uid() = user_id);
```

```
CREATE POLICY "Users can insert own ratings"
ON user_ratings FOR INSERT
WITH CHECK (auth.uid() = user_id);
```

```
CREATE POLICY "Users can update own ratings"
ON user_ratings FOR UPDATE
USING (auth.uid() = user_id);
```

```
CREATE POLICY "Users can delete own ratings"
ON user_ratings FOR DELETE
USING (auth.uid() = user_id);
```

-- Function to update updated_at timestamp

```
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $
BEGIN
    NEW.updated_at = NOW();
    RETURN NEW;
END;
$ language 'plpgsql';
```

-- Triggers for updated_at

```
CREATE TRIGGER update_user_profiles_updated_at
BEFORE UPDATE ON user_profiles
FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
```

```
CREATE TRIGGER update_user_ratings_updated_at
BEFORE UPDATE ON user_ratings
FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
```

Document End

This document serves as the complete blueprint for building ClearSkin.

Pass this document to any developer (including Claude Code or another AI coding assistant) along with the demo prototype, and they will have everything needed to build the production version of the platform.

Good luck with your launch! 🚀

Document Version: 1.0

Last Updated: November 6, 2025

Created by: Claude (Anthropic)

For: ClearSkin Platform Development